CONTOUR TRAINING

Dev Cheatsheet Vol. 2

Java Advanced • System Design • Design Patterns • React • Python

Quick-reference cards + MCQ practice for each topic

SECTION 1 — JAVA ADVANCED

Generics • Lambdas • Streams • Optional • Functional Interfaces

Generics — Why & How

T, E, K, V	Common type parameter names (Type, Element, Key, Value)
List	Type-safe list, no casting needed
? extends T	Upper bound — accepts T or subclasses (read-only)
? super T	Lower bound — accepts T or superclasses (write-safe)
Erasure	Generics are compile-time only; removed at runtime

Functional Interfaces

Predicate	test(T) → boolean — for filtering
Function	apply(T) → R — for transforming
Consumer	accept(T) → void — for side-effects
Supplier	get() → T — for providing values
BiFunction	apply(T,U) → R — two inputs

Lambda Syntax — 4 forms

```
// 1. No params
Runnable r = () -> System.out.println("Hello");

// 2. One param (brackets optional)
Predicate<String> empty = s -> s.isEmpty();

// 3. Multiple params
BiFunction<Integer,Integer,Integer> add = (a, b) -> a + b;

// 4. Block body
Function<String,String> shout = s -> {
    String upper = s.toUpperCase();
    return upper + "!";
};
```

Lambdas implement @FunctionalInterface — any interface with exactly one abstract method.

Stream API — Chained Pipeline

```
List<String> names = List.of("Alice","Bob","Charlie","Anna","Dave");

List<String> result = names.stream()
    .filter(n -> n.startsWith("A"))   // intermediate
    .map(String::toUpperCase)          // intermediate
    .sorted()                          // intermediate
    .collect(Collectors.toList());     // terminal

// Result: [ALICE, ANNA]

// Common terminals
long count  = names.stream().count();
Optional<String> first = names.stream().findFirst();
int total = List.of(1,2,3).stream().mapToInt(i->i).sum();
String joined = names.stream().collect(Collectors.joining(", "));
```

Streams are LAZY — intermediates don't run until a terminal is called. Never reuse a stream.

Optional — Avoid NullPointerException

Optional.of(x)	Wraps x — throws if x is null
Optional.ofNullable(x)	Wraps x — empty if null
Optional.empty()	Creates empty Optional
.isPresent()	Returns true if value exists
.get()	Gets value — throws if empty
.orElse(default)	Returns value or default
.orElseGet(() -> x)	Lazy default via supplier
.ifPresent(fn)	Runs fn only if value present
.map(fn)	Transforms value if present

Method References — 4 Types

ClassName::staticMethod	Math::sqrt (static)
instance::method	System.out::println
ClassName::instanceMethod	String::toUpperCase
ClassName::new	ArrayList::new (constructor)

• Method refs are shorthand for lambdas that just call a method.
• n -> n.toUpperCase() ≡ String::toUpperCase

Java Advanced — MCQ Practice

Q1. What does this stream pipeline return?

A) [ALICE]
B) [ALICE, ANNA] ✓
C) [Alice, Anna]
D) [ANNA, ALICE]

Answer: B) [ALICE, ANNA]
filter keeps names starting with A. map uppercases. sorted alphabetises. Result: [ALICE, ANNA].

Q2. Which functional interface takes a value and returns a boolean?

A) Function
C) Predicate ✓
B) Consumer
D) Supplier

Answer: C) Predicate
Predicate has method test(T t) -> boolean. Used for filtering in streams.

Q3. What is type erasure?

A) Generics are enforced at runtime
C) A way to delete unused types
B) Generic type info is removed at compile time and replaced with Object ✓
D) JVM caches generic types for performance

Answer: B) Generic type info is removed at compile time and replaced with Object
Type erasure: compiler checks generics but JVM only sees Object. No List.class at runtime.

Q4. What is the difference between map() and flatMap() in streams?

A) They are identical
C) flatMap() filters elements
B) map() transforms each element 1-to-1; flatMap() flattens nested streams into one ✓
D) map() is a terminal operation

Answer: B) map() transforms each element 1-to-1; flatMap() flattens nested streams into one
flatMap: List> -> Stream. map would give Stream>.

SECTION 2 — SYSTEM DESIGN BASICS

Scalability • Load Balancing • Databases • Caching • Microservices

Scalability Patterns

Vertical Scaling	Bigger machine — more CPU/RAM. Limited ceiling.
Horizontal Scaling	More machines — distribute load. Preferred for web.
Sharding	Split DB rows across multiple DBs by key range/hash.
Replication	Copy data across multiple nodes for read scaling + failover.
CDN	Cache static assets near users (images, JS, CSS).
Async / Queues	Decouple services with message queues (Kafka, RabbitMQ).

Load Balancing

Round Robin	Requests rotate across servers evenly.
Least Conn.	Sends to server with fewest active connections.
IP Hash	Same client IP always hits same server (sticky session).
L4 LB	Routes by TCP/IP — fast, no content inspection.
L7 LB	Routes by HTTP content (URL, headers) — smarter.
Health checks	LB removes unhealthy servers automatically.

SQL vs NoSQL — When to Use Which

SQL (relational)	Structured data, ACID transactions, complex joins. e.g. user accounts, orders.
NoSQL (document)	Flexible schema, JSON docs. MongoDB. e.g. product catalogs, content.
NoSQL (key-value)	Ultra-fast lookups. Redis, DynamoDB. e.g. sessions, caching.
NoSQL (columnar)	Wide columns, analytics. Cassandra. e.g. time-series, IoT.
CAP Theorem	Can only guarantee 2 of 3: Consistency, Availability, Partition Tolerance.
ACID	Atomicity, Consistency, Isolation, Durability — SQL guarantee.
BASE	Basically Available, Soft state, Eventually consistent — NoSQL style.

Caching Strategy Reference

Cache-aside	App checks cache first. On miss: load from DB, store in cache. (Most common)
Write-through	Write to cache AND DB simultaneously. Consistent but slower writes.
Write-back	Write to cache only; flush to DB asynchronously. Fast but risk of loss.
Read-through	Cache sits in front of DB; cache handles loading automatically.
Eviction: LRU	Least Recently Used — evicts oldest accessed item first.
Eviction: LFU	Least Frequently Used — evicts least accessed item first.
Redis vs Memcached	Redis: persistent, data structures, pub/sub. Memcached: simple, fast.

System Design — MCQ Practice

Q5. What is the CAP theorem?

A) A database can be Consistent, Available, and Partition-tolerant simultaneously

B) A distributed system can only guarantee 2 of: Consistency, Availability, Partition Tolerance ✓

C) A caching strategy for distributed systems

D) A method for horizontal scaling

Answer: B) A distributed system can only guarantee 2 of: Consistency, Availability, Partition Tolerance

CAP: pick 2. CP = consistent + partition tolerant (MongoDB). AP = available + partition tolerant (Cassandra).

Q6. What is the main difference between horizontal and vertical scaling?

A) Horizontal = faster CPU, Vertical = more servers

B) Horizontal = add more servers, Vertical = upgrade existing server hardware ✓

C) They are the same

D) Horizontal scaling is only for databases

Answer: B) Horizontal = add more servers, Vertical = upgrade existing server hardware

Vertical has a hardware ceiling. Horizontal scales indefinitely — preferred for web apps.

Q7. Which caching strategy is most commonly used in backend applications?

A) Write-back

B) Write-through

C) Cache-aside (Lazy Loading) ✓

D) Read-through

Answer: C) Cache-aside (Lazy Loading)

Cache-aside: check cache → miss → load DB → store in cache. App controls caching logic. Most flexible.

Q8. What is database sharding?

A) Creating read replicas of a database

B) Splitting database rows across multiple database instances by a shard key ✓

C) Encrypting database records

D) Backing up the database

Answer: B) Splitting database rows across multiple database instances by a shard key

Sharding = horizontal partitioning. Each shard holds a subset of data. Improves write scalability.

SECTION 3 — DESIGN PATTERNS

Creational • Structural • Behavioural — with code sketches

Pattern Categories

Creational	How objects are created. Singleton, Factory, Builder, Prototype.
Structural	How objects are composed. Adapter, Decorator, Facade, Proxy.
Behavioural	How objects communicate. Observer, Strategy, Command, Iterator.

• Patterns are solutions to recurring OOP problems — not copy-paste code.

Quick Pattern Reference

Singleton	One instance only — DB connection, config, logger.
Factory	Create objects without specifying exact class.
Builder	Step-by-step construction of complex objects.
Observer	Event system — subject notifies all subscribers.
Strategy	Swap algorithm at runtime — payment methods.
Decorator	Add behaviour to object without changing class.
Facade	Simplified interface over a complex subsystem.
Adapter	Make incompatible interfaces work together.

Singleton Pattern

```
public class Config {
    private static Config instance;  // single instance

    private Config() {}              // private constructor

    public static Config getInstance() {
        if (instance == null) {
            instance = new Config();
        }
        return instance;             // always same object
    }
}
// Usage: Config.getInstance().getValue("key");
```

Thread-safe version: use 'synchronized' or initialize in static block. Or use enum Singleton.

Observer Pattern

```
interface Observer { void update(String event); }

class EventBus {
    private List<Observer> subs = new ArrayList<>();
    public void subscribe(Observer o) { subs.add(o); }
    public void publish(String event) {
        for (Observer o : subs) o.update(event);
    }
}

// Usage
EventBus bus = new EventBus();
bus.subscribe(e -> System.out.println("Email: " + e));
bus.subscribe(e -> System.out.println("SMS: " + e));
bus.publish("Order placed");   // notifies all
```

Observer = pub/sub. Used in event systems, UI listeners, message queues.

Strategy Pattern

```
interface SortStrategy { void sort(int[] arr); }

class BubbleSort implements SortStrategy {
    public void sort(int[] arr) { /* bubble logic */ }
}
class MergeSort implements SortStrategy {
    public void sort(int[] arr) { /* merge logic */ }
}

class Sorter {
    private SortStrategy strategy;
    public Sorter(SortStrategy s) { this.strategy = s; }
    public void sort(int[] arr) { strategy.sort(arr); }
}

// Swap algorithm at runtime:
Sorter s = new Sorter(new MergeSort());
s.sort(data);
```

Strategy eliminates if/else chains for algorithm selection. Follows Open/Closed Principle.

Design Patterns — MCQ Practice

Q9. Which pattern ensures only one instance of a class exists?

A) Factory
B) Prototype
C) Builder
D) Singleton ✓

Answer: D) Singleton
Singleton: private constructor + static getInstance(). Used for shared resources like DB connections.

Q10. The Observer pattern is best described as:

A) A way to create objects without a constructor
B) A subject that notifies all registered observers when its state changes ✓
C) Wrapping an object to add new behaviour
D) Adapting one interface to match another

Answer: B) A subject that notifies all registered observers when its state changes
Observer = event/listener model. Used in UI events, message brokers, reactive programming.

Q11. Which pattern category does Adapter belong to?

A) Creational
B) Behavioural
C) Structural ✓
D) Functional

Answer: C) Structural
Structural patterns deal with object composition. Adapter makes incompatible interfaces work together.

Q12. When would you use the Builder pattern?

A) When you need only one instance of a class

B) When constructing complex objects step by step with many optional parameters ✓

C) When you want to notify multiple objects of an event

D) When you want to swap algorithms at runtime

Answer: B) When constructing complex objects step by step with many optional parameters

Builder avoids constructors with 10+ parameters. Example: HttpRequest.Builder in Java 11.

SECTION 4 — REACT & FRONTEND BASICS

Components • Hooks • State • Props • Lifecycle • Performance

React Core Concepts

Component	Reusable UI unit — function that returns JSX.
JSX	HTML-like syntax compiled to React.createElement() calls.
Props	Read-only data passed from parent to child. Never mutate props.
State	Internal mutable data. Changes trigger re-render. Use useState.
Virtual DOM	React's in-memory DOM. Diffs with real DOM — only updates changes.
Key prop	Unique ID for list items — helps React track changes efficiently.
Controlled	Input value driven by state. Uncontrolled = DOM manages value.

React Hooks Reference

useState	Local state: const [val, setVal] = useState(init)
useEffect	Side effects — runs after render. [] = once, [dep] = on change.
useContext	Consume React context — avoids prop drilling.
useRef	Mutable ref — DOM access or persist value without re-render.
useMemo	Cache expensive calculation — recalculate only when deps change.
useCallback	Cache function reference — avoids child re-renders.
useReducer	Complex state logic — like useState but with reducer fn.
Custom hooks	Functions starting with 'use' that reuse stateful logic.

useState + useEffect Pattern

```
import { useState, useEffect } from 'react';

function UserCard({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch(`/api/users/${userId}`)
      .then(res => res.json())
      .then(data => {
        setUser(data);
        setLoading(false);
      });
  }, [userId]);           // re-run when userId changes

  if (loading) return <p>Loading...</p>;
  return <h2>{user.name}</h2>;
}
```

Empty array [] = componentDidMount. [dep] = componentDidUpdate. cleanup return = componentWillUnmount.

Performance Optimisation

React.memo	HOC — skips re-render if props unchanged.
useMemo	Cache computed value — expensive calculations.
useCallback	Cache function — prevents child component re-renders.
Lazy loading	React.lazy + Suspense — code-split routes/components.
List keys	Always provide stable unique keys for mapped lists.
Avoid anon fns	Inline arrow functions in JSX recreate on every render.

CSS Fundamentals Quick Ref

Box model	content → padding → border → margin
Flexbox	display:flex; justify-content; align-items; gap
Grid	display:grid; grid-template-columns; grid-gap
Position	static|relative|absolute|fixed|sticky
z-index	Stacking order — higher = in front. Needs position set.
rem vs em	rem = relative to root font. em = relative to parent.
Media query	@media (max-width: 768px) { ... }

React & Frontend — MCQ Practice

Q13. What triggers a re-render in a React functional component?

A) Changing a variable inside the component

B) Calling setState or returning new state from useState ✓

C) Calling console.log

D) Importing a new module

Answer: B) Calling setState or returning new state from useState

Only state and prop changes trigger re-renders. Regular variable changes are ignored by React.

Q14. What is the purpose of the dependency array in useEffect?

A) It lists all useState variables

B) It controls WHEN the effect runs — only when listed values change ✓

C) It speeds up the component

D) It prevents the component from rendering

Answer: B) It controls WHEN the effect runs — only when listed values change

[] = run once on mount. [id] = run when id changes. No array = run on every render.

Q15. What is prop drilling and how is it solved?

A) Passing props too fast — solved with useMemo

B) Passing props through many intermediate components — solved with Context or state management ✓

C) A bug where props are undefined — solved with default props

D) Using too many props — solved by using fewer components

Answer: B) Passing props through many intermediate components — solved with Context or state management

Prop drilling = passing data through 5+ components just to reach a deeply nested one. Use useContext or Redux.

Q16. What does React.memo do?

A) Memoizes state values

B) Caches expensive calculations

C) Wraps a component to skip re-render if its props haven't changed ✓

D) Stores function references

Answer: C) Wraps a component to skip re-render if its props haven't changed

React.memo = shallow prop comparison. If same props, skip re-render. Use for pure/expensive components.

SECTION 5 — PYTHON FUNDAMENTALS

Syntax • Data Structures • OOP • Comprehensions • Error Handling

Python Data Types & Structures

list []	Ordered, mutable. a = [1,2,3]. a.append(4). a[0].
tuple ()	Ordered, immutable. t = (1,2). Use for fixed data.
dict {}	Key-value. d = {'a':1}. d['a']. d.get('b', default).
set {}	Unordered, unique. s = {1,2,3}. s.add(4). s & s2.
str	Immutable. f-strings: f'Hello {name}'. s.split(), s.strip().
None	Python's null. Check: if x is None (not ==).
Type hints	def fn(x: int) -> str: — not enforced but good practice.

Python OOP

class	class Dog: pass
__init__	Constructor: def __init__(self, name): self.name = name
self	Refers to instance — like 'this' in Java
Inheritance	class Poodle(Dog): — extends Dog
super()	super().__init__(name) — call parent constructor
__str__	def __str__(self): return f'Dog({self.name})'
@property	Getter decorator — access method like an attribute
@staticmethod	No self — belongs to class not instance
@classmethod	cls param — factory methods

Comprehensions — Python Superpower

```
# List comprehension
squares = [x**2 for x in range(10)]            # [0,1,4,9,...,81]

# With condition
evens = [x for x in range(20) if x % 2 == 0]  # [0,2,4,...,18]

# Dict comprehension
word_len = {w: len(w) for w in ['hi','hello','hey']}
# {'hi': 2, 'hello': 5, 'hey': 3}

# Set comprehension
unique_lens = {len(w) for w in ['hi','hello','hey']}  # {2,3,5}

# Generator expression (lazy — doesn't build list in memory)
total = sum(x**2 for x in range(1000000))
```

Comprehensions are faster and more Pythonic than equivalent for-loops.

Exception Handling

try/except	try: risky() except ValueError as e: handle(e)
finally	Runs always — close files, release resources.
raise	raise ValueError('bad input') — throw exception
Custom exception	class AppError(Exception): pass
else block	Runs if no exception was raised in try block.
with statement	with open('f.txt') as f: — auto-closes on exit (context manager)

Useful Built-ins & Patterns

enumerate	for i, v in enumerate(lst): — index + value
zip	for a,b in zip(l1, l2): — pair two lists
map/filter	map(fn, lst) filter(fn, lst) — prefer comprehensions
sorted	sorted(lst, key=lambda x: x.age, reverse=True)
***args**	def fn(*args): — variable positional args as tuple
****kwargs**	def fn(**kwargs): — variable keyword args as dict
__name__	if __name__ == '__main__': — run only when executed directly

Python — MCQ Practice

Q17. What is the output of [x*2 for x in range(4)]?

A) [1,2,3,4]
C) [2,4,6,8]
B) [0,2,4,6] ✓
D) [0,1,2,3]

Answer: B) [0,2,4,6]
range(4) = 0,1,2,3. Multiplied by 2: 0,2,4,6.

Q18. What is the difference between a list and a tuple in Python?

A) Lists are faster
C) Tuples can hold more items
B) Lists are mutable (can be changed); tuples are immutable (cannot be changed) ✓
D) Lists use {} and tuples use []

Answer: B) Lists are mutable (can be changed); tuples are immutable (cannot be changed)
list = [], mutable. tuple = (), immutable. Use tuples for fixed data — also hashable so can be dict keys.

Q19. What does the 'self' parameter in a Python class method refer to?

A) The class itself
C) The current instance of the class ✓
B) The parent class
D) A static reference to the method

Answer: C) The current instance of the class
self = 'this' in Java. Python requires explicit self as first param. Not a keyword — just convention.

Q20. What does the 'with' statement do when opening a file?

A) Opens a file in write mode only

B) Automatically closes the file when the block exits, even if an exception occurs ✓

C) Locks the file for exclusive access

D) Creates a copy of the file

Answer: B) Automatically closes the file when the block exits, even if an exception occurs

with = context manager. Calls __enter__ and __exit__. No need for f.close() — clean and safe.

ANSWER KEY — ALL SECTIONS

1-B	2-C	3-B	4-B	5-B
6-B	7-C	8-B	9-D	10-B
11-C	12-B	13-B	14-B	15-B
16-C	17-B	18-B	19-C	20-B